

Session : Principale
Régime : RM
Enseignant : Sofiane HACHICHA
Classe(s) : CPI2
Barème : 9 + 11

Date : 25/05/2023
Matière : Programmation Java 2
Durée : 90 minutes
Documents : Non autorisés
Nombre des pages : 4

Examen

➤ *Il sera tenu compte de la présentation de la copie (1 point bonus).*

Exercice 1

On souhaite développer une application Java en mode console pour gérer une liste de contacts. Chaque contact aura un nom, un numéro de téléphone et une adresse e-mail.

Les fonctionnalités attendues de l'application sont les suivantes :

- Ajouter un nouveau contact en saisissant son nom, son numéro de téléphone et son adresse e-mail.
- Afficher la liste de tous les contacts enregistrés.
- Rechercher un contact par son nom et afficher son numéro de téléphone et son adresse e-mail. Il est supposé que chaque contact possède un nom unique.
- Supprimer un contact de la liste en saisissant son nom.

Pour réaliser cela, il est nécessaire d'utiliser la sérialisation pour enregistrer et charger la liste des contacts à partir d'un fichier accessible via un objet Path. De plus, l'utilisation d'une collection de type ArrayList pour stocker les contacts est obligatoire.

Afin de faciliter la gestion des contacts, il est recommandé de créer une classe **Contact** avec des attributs pour le nom, le numéro de téléphone et l'adresse e-mail, ainsi que des méthodes getters et setters pour chaque attribut, ainsi qu'une méthode toString().

Pour simplifier l'enregistrement et la lecture des contacts à partir du fichier de sérialisation, il est recommandé de créer une classe **ContactManager** qui gère l'ajout, la suppression, la recherche et l'affichage des contacts.

Pour faciliter la gestion des entrées utilisateur, il est obligatoire d'utiliser la classe Scanner de Java pour lire les entrées de l'utilisateur à partir de la console.

Dans la méthode principale (main), il est nécessaire de présenter un menu offrant les choix suivants :

1. Ajouter un contact
2. Rechercher un contact par son nom
3. Supprimer un contact
4. Afficher la liste des contacts
5. Quitter le programme

Le programme doit continuer à s'exécuter jusqu'à ce que l'utilisateur choisisse de le quitter.

```
import java.io.Serializable;
import java.io.*;
import java.nio.file.*;
import java.util.*;

class Contact implements Serializable {

    String nom;
    int num;
    String email;

    public Contact(String nom, int num, String email) {
        this.nom = nom;
        this.num = num;
        this.email = email;
    }
    public String getNom() {
        return nom;
    }
    public void setNom(String nom) {
        this.nom = nom;
    }
    public int getNum() {
        return num;
    }
    public void setNum(int num) {
        this.num = num;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
    public String toString() {
        return "Contact [nom=" + nom + ", num=" + num + ", email=" + email + "];"
    }
}

class ContactManager {

    private static List<Contact> contacts = new ArrayList<>();

    public static Contact createContact(Scanner scanner) {
        System.out.print("Nom : ");
        String nom = scanner.next();
        System.out.print("Téléphone : ");
        int tel = scanner.nextInt();
        System.out.print("Email : ");
        String email = scanner.next();
        return new Contact(nom, tel, email);
    }

    public static List<Contact> chargerContact (Path FILE_PATH) {
        if (FILE_PATH.toFile().length()>0) {
            try (ObjectInputStream ois = new ObjectInputStream(Files.newInputStream(FILE_PATH))) {
```

```
        contacts = (List<Contact>) ois.readObject();

    } catch (IOException | ClassNotFoundException e) {
        e.printStackTrace();
    }
}

return contacts;
}

public static void sauvgarderContact (Path FILE_PATH) {

try (ObjectOutputStream oos = new ObjectOutputStream(Files.newOutputStream(FILE_PATH))) {
    oos.writeObject(contacts);
    } catch (IOException e) {
        e.printStackTrace();
    }
}

}

public class TestExercice1 {

    private static final Path FILE_PATH = Paths.get("E:", "contacts.bin"); // Juste un exemple.

    public static void main(String[] args) throws IOException {
        if(!Files.exists(FILE_PATH))
            Files.createFile(FILE_PATH);
        Scanner scanner = new Scanner(System.in) ;
        List<Contact> contacts= ContactManager.chargerContact(FILE_PATH);
        boolean quit = false;
        while (!quit) {
            System.out.println("\n Menu :");
            System.out.println("1. Ajouter un contact");
            System.out.println("2. Rechercher un contact par son nom");
            System.out.println("3. Supprimer un contact");
            System.out.println("4. Afficher la liste des contacts");
            System.out.println("5. Quitter le programme");
            System.out.print("Votre choix : ");
            String choice = scanner.next();

            switch (choice) {
                case "1":
                    Contact contact = ContactManager.createContact(scanner);
                    contacts.add(contact);
                    System.out.println("Contact ajouté avec succès !");
                    break;

                case "2":
                    System.out.print("Entrez le nom du contact à rechercher : ");
                    String query = scanner.next();
                    boolean found = false;
                    for (Contact c : contacts) {
                        if (c.getNom().equals(query)) {
                            System.out.println(c);
                            found = true;
                        }
                    }
                }
            }
        }
    }
}
```

```

    }
    if (!found) {
        System.out.println("Aucun contact ne correspond à la recherche.");
    }
    break;
case "3":
    System.out.print("Entrez le nom du contact à supprimer : ");
    String q = scanner.next();
    if( contacts.removeIf( e -> e.getNom().equals(q)))
        System.out.println("Contact supprimé");
    else
        System.out.println("Aucun contact ne correspond à la recherche.");
    break;
case "4":
    if (contacts.isEmpty()) {
        System.out.println("Il n'y a pas de contacts à afficher.");
    } else {
        for (Contact c : contacts) {
            System.out.println(c);
        }
    }
    break;
case "5":
    quit = true;
    break;
default:
    System.out.println("Choix invalide.");
    break;
}
}

ContactManager.sauvgarderContact(FILE_PATH);

}
}

```

Exercice 2

On souhaite développer une application Java en mode graphique en utilisant la bibliothèque Swing. L'application permettra à l'utilisateur de saisir les informations suivantes via une interface simpliste :

- Une URL de connexion via un champ de texte (JTextField).
- Le nom de l'utilisateur se connectant à la base de données via un champ de texte (JTextField).
- Le mot de passe de l'utilisateur via un champ de texte masqué (JPasswordField).

Lorsque l'utilisateur clique sur le bouton "Ouvrir une connexion" après avoir saisi correctement les informations, le programme tentera d'établir une connexion à la base de données. Si la connexion réussit, un message "Connexion établie" sera affiché dans un JLabel situé sous le bouton, et le bouton "Créer une table" sera activé, permettant ainsi à l'utilisateur de créer la

table "**clients**" dans la base de données. En cas d'échec de la connexion, un message "Connexion impossible" sera affiché dans le même JLabel.

Une fois que le bouton "Créer une table" est activé, si l'utilisateur clique dessus, un message "Création réalisée" sera affiché dans le JLabel situé sous le bouton. En conséquence, la table "**clients**" sera ajoutée à la base de données, le bouton "Insérer un client" deviendra actif, tandis que les boutons "Ouvrir une connexion" et "Créer une table" seront désactivés. Si un problème survient lors de la création de la table, un message "Création impossible" sera affiché dans le JLabel situé sous le bouton "Créer une table".

Une fois que le bouton "Insérer un client" est activé, si l'utilisateur clique dessus, deux boîtes de dialogue apparaîtront successivement. La première boîte de dialogue demandera à l'utilisateur de saisir le nom du client, tandis que la deuxième boîte de dialogue lui demandera de saisir l'âge du client. Si les données du client sont saisies avec succès, le client sera ajouté dans la table "**clients**", un message "Insertion réussie" sera affiché dans un JLabel situé sous le bouton, et le bouton "Compter les clients" deviendra actif. Si l'utilisateur ne saisit pas de nom, le client est ajouté avec une valeur nulle pour le nom. Si l'utilisateur saisit un âge qui n'est pas un nombre entier positif, la valeur de l'âge sera nulle. Les clients sont ajoutés à la table "**clients**" de manière cumulative, ce qui signifie que l'utilisateur peut insérer plusieurs clients en cliquant sur le bouton "Insérer un client" à chaque fois.

Une fois que le bouton "Compter les clients" est activé, si l'utilisateur clique dessus, une boîte de dialogue s'affiche, demandant à l'utilisateur de saisir un seuil d'âge minimal souhaité pour les clients. Si le seuil est saisi avec succès (un nombre entier positif), un message, contenant le texte "Nombre de clients = " suivi du nombre de clients insérés dans la table "**clients**" ayant un âge supérieur ou égal au seuil saisi, sera affiché dans le JLabel situé le bouton. En cas de problème, un message "Problème survenu" sera affiché dans le JLabel situé sous le bouton "Compter les clients".

Lorsque l'utilisateur clique sur le bouton "Quitter", la fenêtre de l'application se ferme.

Au démarrage de l'application, seuls les boutons "Ouvrir une connexion" et "Quitter" sont actifs. Il est recommandé d'utiliser les gestionnaires d'agencement appropriés pour organiser les interfaces graphiques. La méthode `setBounds()` ne doit pas être utilisée.

En résumé, l'objectif de cette application est d'établir une connexion à la base de données "**examendb**" en utilisant l'URL suivante : "**jdbc:mysql://localhost:3306/examendb**".

Les informations d'identification pour la connexion sont le nom d'utilisateur "**root**" et le mot de passe "**ising**".

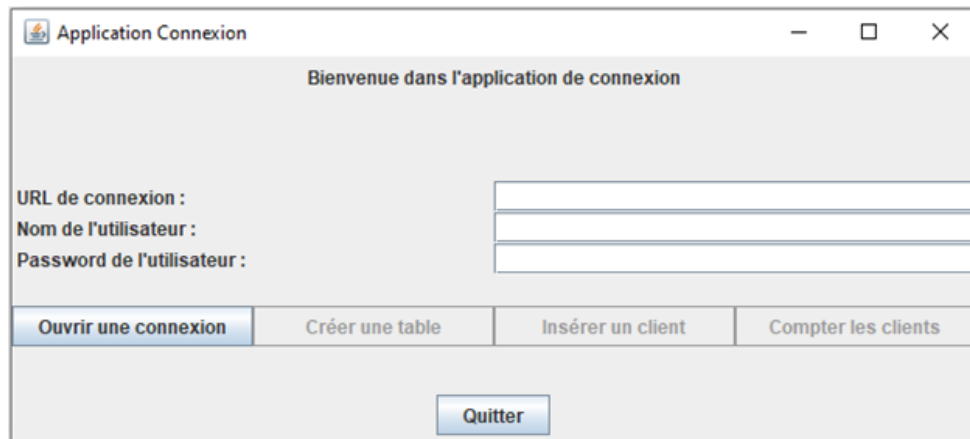
Une table nommée "**clients**" sera créée, comprenant les colonnes suivantes : "**idClient**" (un entier utilisé comme clé primaire avec auto-incrémentation), "**nomClient**" (une chaîne de caractères pouvant être NULL pour stocker les noms des clients) et "**ageClient**" (un entier pouvant être NULL et représentant l'âge des clients).

Les fonctionnalités suivantes peuvent être utilisées pour implémenter les différentes fonctionnalités de l'application décrites précédemment :

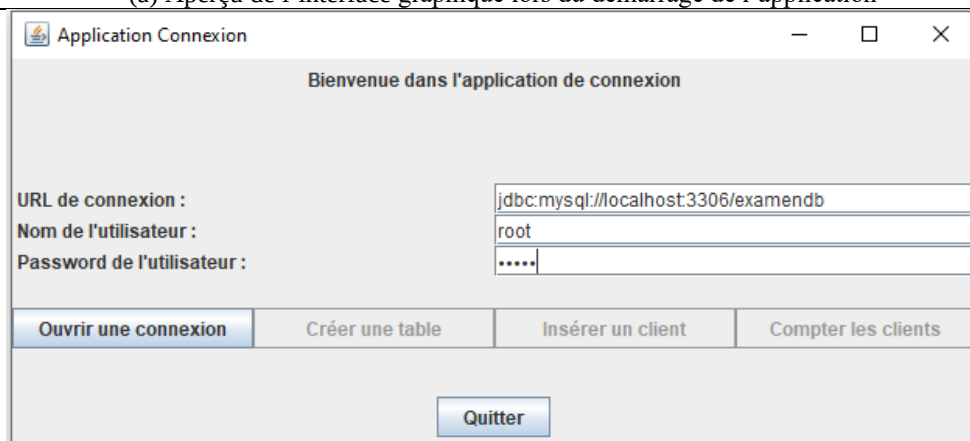
- La méthode `getPassword()` qui renvoie le contenu d'un `JPasswordField` sous forme d'un tableau de caractères.
- La méthode `getText()` qui renvoie le contenu d'un `JTextField` sous forme d'une chaîne de caractères.

- La méthode `setEnabled(boolean b)` pour rendre un composant inactif lorsqu'elle est appelée.
- La commande SQL "DROP TABLE" suivie de la commande "CREATE TABLE" pour supprimer et recréer la table "**clients**".
- Une requête préparée (PreparedStatement) pour insérer un client dans la table "**clients**".

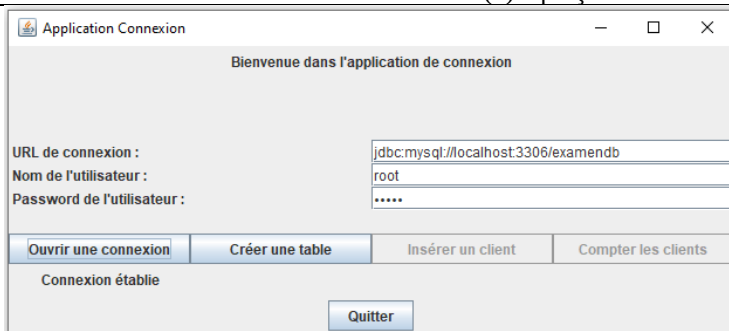
Voici un tableau récapitulatif des différents scénarios d'exécution de l'application pour plus de clarté.



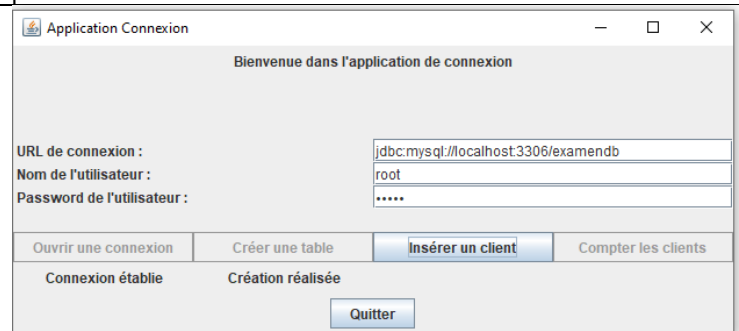
(a) Aperçu de l'interface graphique lors du démarrage de l'application



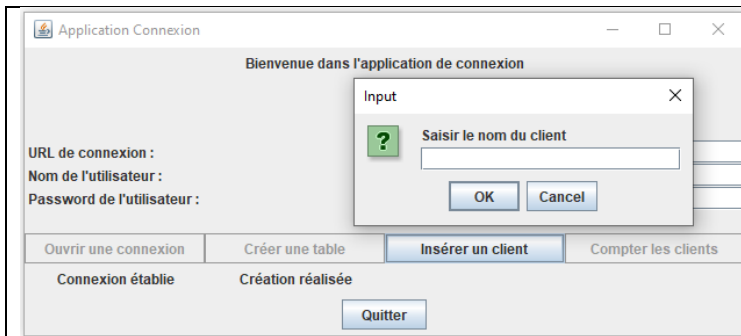
(b) Aperçu de l'interface après avoir saisi les informations de connexion



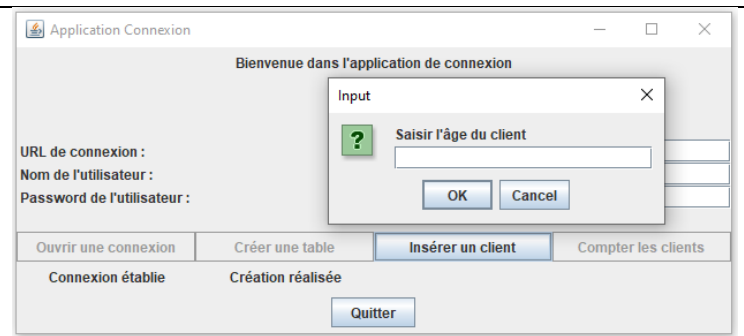
(c) Aperçu de l'interface après avoir cliqué sur le bouton "Ouvrir une connexion"



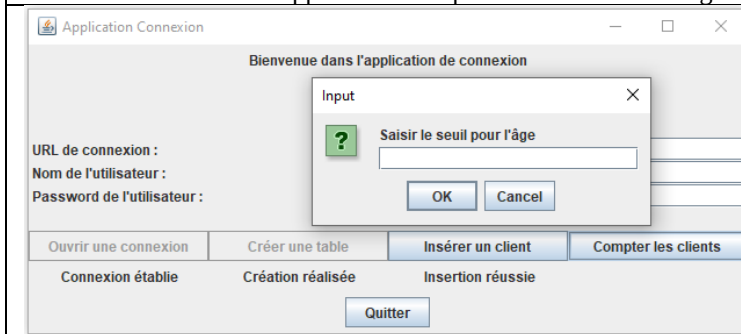
(d) Aperçu de l'interface après avoir cliqué sur le bouton "Créer une table"



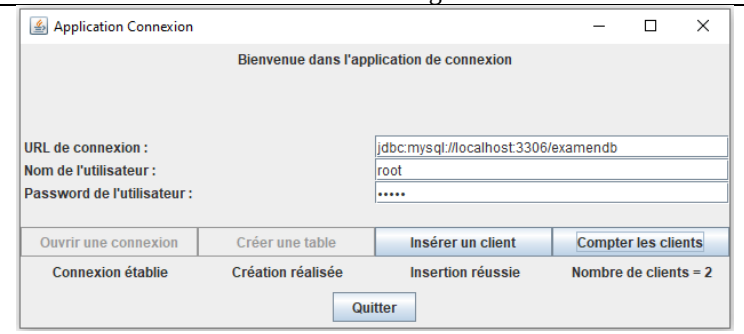
(e1) Aperçu de l'interface après avoir cliqué sur le bouton "Insérer un client" et l'apparition de la première boîte de dialogue



(e2) Aperçu de l'interface après l'apparition de la deuxième boîte de dialogue



(f1) Aperçu de l'interface après avoir cliqué sur le bouton "Compter les clients" et l'apparition d'une boîte de dialogue



(f2) Aperçu de l'interface indiquant le nombre de clients insérés ayant un âge supérieur ou égal au seuil saisi.

Tableau – Scénarios d'exécution de l'application

```
import java.awt.*;
import java.awt.event.*;
import java.sql.*;
import javax.swing.*;

public class TestExercice2 implements ActionListener {
    private JButton b1, b2, b3, b4, b5;
    private JLabel l1, l2, l3, l4;
    private JTextField tf1, tf2;
    private JPasswordField pf1;
    private JPanel p1, p2, p3, p4, p12;
    private JFrame f;
    private static Connection con;
    private static final String DB_URL = "jdbc:mysql://localhost:3306/examendb";
    private static final String DB_USERNAME = "root";
    private static final String DB_PASSWORD = "isimg";

    public TestExercice2() {

        f = new JFrame("Application Connexion");
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        p1 = new JPanel();
        p1.add(new JLabel("Bienvenue dans l'application de connexion"));

        p2 = new JPanel();
        p2.setLayout(new GridLayout(4, 2));
        tf1 = new JTextField();
        tf2 = new JTextField();
```

```
pf1 = new JPasswordField();
p2.add(new JLabel(" URL de connexion : "));
p2.add(tf1);
p2.add(new JLabel(" Nom de l'utilisateur : "));
p2.add(tf2);
p2.add(new JLabel(" Password de l'utilisateur : "));
p2.add(pf1);

p12 = new JPanel(new GridLayout(2, 1));
p12.add(p1);
p12.add(p2);

f.getContentPane().add(p12, BorderLayout.PAGE_START);

p3 = new JPanel(new FlowLayout());
p3.setLayout(new GridLayout(2, 4));
b1 = new JButton("Ouvrir une connexion");
l1 = new JLabel();
l1.setHorizontalAlignment(JLabel.CENTER);

b2 = new JButton("Créer une table");
l2 = new JLabel();
l2.setHorizontalAlignment(JLabel.CENTER);

b3 = new JButton("Insérer un client");
l3 = new JLabel();
l3.setHorizontalAlignment(JLabel.CENTER);

b4 = new JButton("Compter les clients");
l4 = new JLabel();
l4.setHorizontalAlignment(JLabel.CENTER);

b2.setEnabled(false);
b3.setEnabled(false);
b4.setEnabled(false);
p3.add(b1);
p3.add(b2);
p3.add(b3);
p3.add(b4);
p3.add(l1);
p3.add(l2);
p3.add(l3);
p3.add(l4);

b1.addActionListener(this);
b2.addActionListener(this);
b3.addActionListener(this);
b4.addActionListener(this);

f.getContentPane().add(p3, BorderLayout.CENTER);

p4 = new JPanel();
b5 = new JButton("Quitter");
p4.add(b5);
b5.addActionListener(e -> f.dispose());
f.getContentPane().add(p4, BorderLayout.PAGE_END);
```

```
f.pack();
f.setLocationRelativeTo(null);
f.setVisible(true);
}

public void actionPerformed(ActionEvent e) {
    if (e.getSource() == b1) {
        if (DB_URL.equals(tf1.getText()) && DB_USERNAME.equals(tf2.getText())
            &&
            DB_PASSWORD.equals(String.valueOf(pf1.getPassword())))
            try {
                con = DriverManager.getConnection(DB_URL, DB_USERNAME, DB_PASSWORD);
                l1.setText(" Connexion établie");
                b2.setEnabled(true);
            } catch (SQLException e1) {
                l1.setText(" Connexion impossible");
            }
        else
            l1.setText(" Connexion impossible");
    }

    else if (e.getSource() == b2) {
        b1.setEnabled(false);
        try {
            creerTable(con);
            l2.setText(" Création réalisée");
            b3.setEnabled(true);
            b2.setEnabled(false);
        } catch (SQLException e1) {
            l2.setText(" Création impossible");
        }
    }

    else if (e.getSource() == b3) {
        int x;
        l3.setText("");
        String nom = JOptionPane.showInputDialog(null, "Saisir le nom du client");
        try {
            x = Integer.parseInt(JOptionPane.showInputDialog(null, "Saisir l'âge du client"));
            if (x < 0)
                x = -1;
        } catch (NumberFormatException e0) {
            x = -1;
        }

        try {
            ParametreTable(con, nom, x);
            l3.setText(" Insertion réussie");
            b4.setEnabled(true);
        } catch (SQLException e1) {
            l3.setText(" Insertion impossible");
        }
    }
}
```

```
        else if (e.getSource() == b4) {
            try {
                int v = Integer.parseInt(JOptionPane.showInputDialog(null, "Saisir le seuil pour l'âge"));
                if (v < 0)
                    throw new Exception();
                int res = ResultTable(con, v);
                l4.setText(" Nombre de clients = " + res);
            } catch (Exception e1) {
                l4.setText(" Problème survenue");
            }
        }
        else if (e.getSource() == b5) {
            f.dispose();
        }
    }

    public static void creerTable(Connection con) throws SQLException {
        try (Statement stmt = con.createStatement()) {
            stmt.executeUpdate("DROP TABLE IF EXISTS table_client");
            stmt.executeUpdate("CREATE TABLE table_client (id INT
AUTO_INCREMENT PRIMARY KEY, nom VARCHAR(100), age INT)");
        }
    }

    public static void ParametreTable(Connection con, String s, int i) throws SQLException {
        String req = "Insert into table_client (nom, age) values (?,?)";
        try (PreparedStatement st = con.prepareStatement(req)) {
            st.setString(1, s);
            if (i >= 0)
                st.setInt(2, i);
            else
                st.setNull(2, Types.INTEGER);
            st.executeUpdate();
        }
    }

    public static int ResultTable(Connection con, int val) throws SQLException {
        try (PreparedStatement stmt = con.prepareStatement("SELECT id FROM
table_client WHERE age > ?")) {
            int i = 0;
            stmt.setInt(1, val);
            ResultSet rs = stmt.executeQuery();
            while (rs.next()) {
                i++;
            }
            return i;
        }
    }

    public static void main(String[] args) {

        SwingUtilities.invokeLater( () -> new TestExercice2() );
    }
}
```